ACADEMIC CENTRAL

PRACTICE EXAMINATION

NC(V) LEVEL 4

TECHNICAL PROGRAMMING

PRACTICE EXAM 2 · MARKING GUIDELINE

MARKS: 100

Marking guideline for the Academic Central NC(V) Level 4 Technical Programming practice exam 2.

Written by Academic Central for practice, following the NC(V) subject guidelines for NC(V) Level 4. It is not a Department of Higher Education and Training marking guideline.

How to mark: each ✓ is one mark. Award marks for correct logic even if there are small syntax slips. Accept any correct equivalent Python code and variable names.

SECTION A

QUESTION 1

1.1 1.1.1 C ✓ 1.1.2 A ✓ 1.1.3 B ✓ 1.1.4 D ✓ 1.1.5 A ✓ 1.1.6 C ✓ 1.1.7 C ✓ 1.1.8 A ✓ 1.1.9 D ✓ 1.1.10 B ✓ (10)

1.2 1.2.1 True ✓ 1.2.2 False ✓ 1.2.3 True ✓ 1.2.4 True ✓ 1.2.5 True ✓ 1.2.6 False ✓ 1.2.7 True ✓ 1.2.8 True ✓ 1.2.9 True ✓ 1.2.10 False ✓ (10)

1.3 1.3.1 E ✓ 1.3.2 B ✓ 1.3.3 G ✓ 1.3.4 C ✓ 1.3.5 I ✓ 1.3.6 J ✓ 1.3.7 F ✓ 1.3.8 H ✓ 1.3.9 A ✓ 1.3.10 D ✓ (10)

TOTAL SECTION A: [30]

SECTION B

QUESTION 2

2.1 A C++ compiler translates the whole program into machine code before it runs, producing an executable ✓✓; the Python interpreter translates the source to bytecode and runs it line by line each time the program runs, so there is no separate compile step and errors appear while running ✓✓ (4)

2.2 Two inputs with float() ✓✓, speed = distance / time ✓, rounded to 2 decimals ✓, output with unit ✓ (5)

```
distance = float(input("Distance travelled (m): "))
time = float(input("Time taken (s): "))
speed = distance / time
print(f"Speed: {speed:.2f} m/s")
```

2.3 The conversion is placed in a try block; if the text is not a number a ValueError is raised and the except block shows a message instead of crashing ✓✓. Code: try / except ✓, loop or message to ask again ✓ (4)

```
while True:
    try:
        distance = float(input("Distance travelled (m): "))
        break
    except ValueError:
        print("Please type a number.")
```

[13]

QUESTION 3

3.1 if value < 30 ✓ Turn left ✓; elif value <= 70 ✓ Straight ✓; else Turn right ✓ (5)

```
if light < 30:
    print("Turn left")
elif light <= 70:
    print("Straight")
else:
    print("Turn right")
```

3.2.1 3 22 ✓✓ (values 9, 7 and 6 are greater than 5) (2)

3.2.2 List stored in a variable ✓, index starts at 0 ✓, while index < len(...) ✓, same if-statement using values[index] ✓, index += 1 ✓ (5)

```
values = [4, 9, 2, 7, 6]
count = 0
total = 0
i = 0
while i < len(values):
    if values[i] > 5:
        total += values[i]
        count += 1
    i += 1
print(count, total)
```

3.3 Counter for tries ✓, loop limited to three tries ✓, input inside loop ✓, compare with "robot4" ✓, granted + break ✓, "Locked" after three wrong tries ✓ (6)

```
tries = 0
granted = False
while tries < 3:
    password = input("Password: ")
    tries += 1
    if password == "robot4":
        granted = True
        break
if granted:
    print("Access granted")
else:
    print("Locked")
```

[18]

QUESTION 4

4.1 def with parameter ✓, empty-list check ✓ returning 0 ✓, sum and len ✓, return average ✓ (5)

```
def average(values):
    if len(values) == 0:
        return 0
    return sum(values) / len(values)
```

4.2.1 Start with the first value and index 0 ✓, loop with index ✓, compare ✓, update value and index ✓, print both (31.0 at hour 3) ✓ (5)

```
highest = temps[0]
hour = 0
for i in range(1, len(temps)):
    if temps[i] > highest:
        highest = temps[i]
        hour = i
print(highest, hour)      # 31.0 3
```

4.2.2 New empty list ✓, loop with condition > 25 ✓, append ✓ (or a correct list comprehension) (3)

```
hot = []
for t in temps:
    if t > 25:
        hot.append(t)
# or: hot = [t for t in temps if t > 25]
```

4.3.1 battery["R4"] = 100 ✓ (1)

4.3.2 Loop through keys and values ✓✓, condition < 20 ✓, print the name ✓ (prints R2) (4)

```
for name, level in battery.items():
    if level < 20:
        print(name)
```

[18]

QUESTION 5

5.1 class Motor ✓, __init__ with self and name ✓, speed set to 0 ✓, set_speed method ✓, range check 0–255 ✓, assign speed ✓, error message ✓, stop method setting speed to 0 ✓ (8)

```
class Motor:
    def __init__(self, name):
        self.name = name
        self.speed = 0

    def set_speed(self, value):
        if 0 <= value <= 255:
            self.speed = value
        else:
            print("Invalid speed")

    def stop(self):
        self.speed = 0
```

5.2 left = Motor("left") ✓; left.set_speed(180) ✓; left.stop() ✓ (3)

5.3 Any TWO ✓✓: each motor / sensor is an object with its own data, so code is reused instead of copied; easier to read and maintain; data is protected inside the object (encapsulation); new robots can reuse or inherit classes (2)

5.4 A list stores items in order and each item is found by its position (index) ✓✓; a dictionary stores key–value pairs and each value is found by its key ✓✓; lists suit sequences of readings, dictionaries suit looking up data by name ✓✓. Use a dictionary ✓, because each sensor name (key) is linked to its pin number (value), e.g. {"ultrasonic": 7, "ldr": 0} ✓ (8)

[21]

TOTAL SECTION B: [70]

TOTAL: 100

END OF MARKING GUIDELINE